

Generative AI End-Term Project

Dual Text RAG and Multimodal RAG

STAI345 - Generative AI | Vijaybhoomi University
VU Student Handbook as the knowledge source

1. Problem Statement

The objective was to build and evaluate a local Retrieval-Augmented Generation (RAG) system using the official VU Student Handbook. Two lightweight language models were compared in the same RAG pipeline, sharing one retriever, so that any difference in answer quality could be attributed to generation rather than retrieval. The project was then extended from text-only RAG to Multimodal RAG so that the offence table and forms - which extract into garbled, column-flattened text - could be read directly as images by a vision-language model.

2. Approach

The project was completed in two phases:

- 1 **Phase 1** - Build one common text RAG pipeline (shared vector index) and run it with SmoLM2-135M and Llama 3.2-3B.
- 2 Evaluate both models on the same 30 handbook questions, using the same retrieval configuration and evaluation code.
- 3 Analyze retrieval quality, answer quality, faithfulness, relevancy, correctness and latency.
- 4 **Phase 2** - Add a vision-language model (gemma3:4b) so table images (pre-cropped at 300 DPI) can be combined with retrieved text.
- 5 Evaluate Phase 2 on 12 visual questions targeting the offence table, the leave form and the amendment record.
- 6 Add a structured comparison of Phase 1 vs Phase 2 to see where the image actually helps.

3. Models and Main Components

Component	Used in the project
Small text LLM	SmoLM2-135M
Larger text LLM	Llama 3.2-3B Instruct
Judge model	gemma3:4b (scores Faithfulness / Relevancy / Correctness)
Multimodal model (VLM)	gemma3:4b - also used as the judge, but a different family from both generators
Embeddings	nomic-embed-text (with search_query: / search_document: task prefixes)
Vector database	ChromaDB (cosine similarity)
Runtime	Ollama (local, GPU-accelerated) for every model - generation, judging and vision
Backend	Python modules in src/ for document processing, retrieval, generation and evaluation

Both text models used the same handbook, embeddings, vector store, retrieval method and questions. This kept the model comparison controlled: any gap in generation-quality metrics can be attributed to the model, not to what was retrieved.

4. Directory Structure

```
GenAI_RAG_StudentHandbook/  
|-- src/  
| |-- config.py, document.py, embeddings.py, vector_store.py  
| |-- llm.py, rag.py, evaluation.py  
| |-- multimodal.py, multimodal_evaluation.py  
|-- data/  
| |-- handbook.pdf, pages.json  
| |-- pages/*.png (full page renders, 200 DPI)  
| |-- tables/*.png (pre-made table crops, 300 DPI)  
| `-- chroma/ (vector index)  
|-- notebooks/  
| |-- 01_document_processing.ipynb  
| |-- 02_dual_rag.ipynb  
| |-- 03_evaluation.ipynb  
| `-- 04_multimodal_rag.ipynb  
|-- eval/ (text_qa.json, visual_qa.json)  
|-- diagrams/, results/  
`-- environment.yml, README.md
```

5. Phase 1 - Dual Text RAG

5.1 Document Processing

The handbook PDF was parsed page by page with PyMuPDF. The repeated footer, page numbers and table-of-contents dot leaders were stripped, since a raw dot-leader run was found to blow up the embedding model's token count on page 3. Two to three pages with no extractable text (the cover, etc.) were given a short placeholder sentence. Each page was treated as one chunk - page-level chunking, so that one chunk maps to one page and one page image, which lets Phase 2 reuse the same index. All 31 chunks (30 pages plus one front-matter record carrying the approval date) were embedded with nomic-embed-text and stored in a single Chroma collection.

5.2 Retrieval and Generation

The same retrieve() function and the same top-k=4 setting was used for both models. The retrieved context was passed to each model with an identical prompt template and temperature=0. This separates retrieval from generation and makes it possible to identify whether an error comes from retrieval or from the language model.

5.3 Evaluation Method

Thirty handbook-based questions were used for both models (facts, numeric lookups, procedures, multi-hop questions, and four questions whose answer is not in the handbook, to test refusal behaviour). Each model was evaluated independently, using the same questions and the same retrieval configuration.

- **Context Precision** - fraction of retrieved pages that are actually relevant (gold pages).
- **Context Recall** - fraction of the required gold pages that were retrieved.
- **MRR (Mean Reciprocal Rank)** - how early the first relevant page appears in the ranking.
- **Faithfulness** - is every claim in the answer supported by the retrieved context? Scored by an LLM judge (gemma3:4b).
- **Relevancy** - does the answer address what the question actually asked? Scored by the judge.
- **Correctness** - does the answer match the reference answer's meaning? Scored by the judge.
- **Latency** - time taken to generate an answer.

Faithfulness, Relevancy and Correctness are scored by an LLM judge (gemma3:4b), asked one focused question per metric rather than all three in a single combined prompt - a combined prompt was tested first and gave unreliable, templated scores from the judge model. The judge is a different model family from both generators, so it is not grading its own answers.

5.4 Phase 1 Results

Metric	SmolLM2-135M	Llama 3.2-3B
Questions	30	30
Context Precision	0.250	0.250
Context Recall	0.923	0.923
MRR	0.821	0.821
Faithfulness	0.767	0.833
Relevancy	0.800	0.800
Correctness	0.467	0.900
Average Latency	0.475 s	12.175 s

5.5 Analysis of Phase 1

- Retrieval performance was identical for both models: Context Precision = 0.250, Context Recall = 0.923, MRR = 0.821 - expected, since both query the same index with the same top-k.
- Llama 3.2-3B scored nearly twice as high on Correctness (0.900 vs 0.467) and moderately higher on Faithfulness (0.833 vs 0.767), while Relevancy was identical (0.800) for both models.
- Identical relevancy alongside a large correctness gap indicates SmolLM2 generally stays on-topic but frequently gets facts wrong, rather than misunderstanding the question.
- SmolLM2 was observed to hallucinate on out-of-scope questions instead of refusing - for example, it invented a specific tuition fee, a fabricated Vice Chancellor name ("Dr. Rajiv Sharma"), and a fabricated Wi-Fi password, when asked four questions whose answers are not in the handbook. Llama correctly replied "Not in handbook" on the same four questions.
- SmolLM2 also produced clear factual errors on simple lookups even with the correct context retrieved - for example, stating the university is located in "Karaikal, Karnataka" (fabricated) instead of the correct "Karjat, Raigad".
- SmolLM2 was roughly 25x faster on average (0.475 s vs 12.175 s per question).

5.6 Model Comparison for This Experiment

For this 30-question experiment, Llama 3.2-3B gave substantially stronger answer-quality results on every generation metric, while SmolLM2-135M had a ~25x lower average latency. Since retrieval metrics are identical for both models by construction, this gap is attributable to generation quality - model size and instruction-following - not to what was retrieved. The measured trade-off is therefore: Llama 3.2-3B for accuracy-critical use, SmolLM2-135M for latency-critical / resource-constrained use.

6. Phase 2 - Multimodal RAG

The text-only pipeline was extended so that, if the retrieved page has a pre-made table image in data/tables/, that image is sent together with the retrieved text to a vision-language model (gemma3:4b). The table images (offence table, leave form, amendment record) were cropped once from the PDF at 300 DPI; the pipeline only reads them, it does not re-extract them at query time. If a page has no table image, the VLM falls back to text only, the same context the Phase-1 models would see.

- 1 Retrieve the relevant handbook page(s) using the same text retriever and index as Phase 1.
- 2 Look up the retrieved page's pre-made table image, if one exists.
- 3 Send the retrieved text, and the image if available, to gemma3:4b in a single call.
- 4 Ask the model to answer using the text and, if given, the image.
- 5 Record the answer, whether an image was used, latency, and an LLM-judged correctness score.

6.1 Multimodal Evaluation

ID	Question	Image used	Correctness	Latency
V01	Minimum fine for offence 20.1 (explosives/arms/narcotics/alcohol)?	Yes	1.0	1.53 s
V06	Offence number for the row about failing to check in by 9:00 pm?	Yes	0.0	19.66 s
V09	List two roles credited as developers of handbook v1.0.	No	0.0	12.23 s
V12	Title printed on the front cover, and its dominant colour?	No	0.0	15.75 s

Across all 12 visual questions, 10 had a table image attached and 2 fell back to text only. Average correctness across all 12 was 0.75 and average latency was 15.44 s. V01 shows a correct, image-grounded answer. V06 shows a genuine known defect: the offence table's row for the 9:00 pm check-in rule loses its offence number when the table splits across pages 16-17, and the model guessed the wrong number (20.3) from the image rather than reporting the number as missing. V09 and V12 show the fallback failure mode: pages with no table image (the amendment-record roles list and the cover) rely on the same garbled text extraction that limits the Phase-1 models, so the VLM has no advantage there.

6.2 Multimodal Limitation Observed

The multimodal pipeline's accuracy is bounded by two separate things: whether the correct page is retrieved (a retrieval problem, shared with Phase 1) and whether that page has a pre-made table image available at all (a coverage problem specific to Phase 2). On the 2 of 12 questions with no table image, the VLM performs no better than the text-only pipeline, which confirms that the image, not the model, is doing the real work whenever Phase 2 outperforms Phase 1.

7. Structured Comparison: Phase 1 vs Phase 2

Phase 1 and Phase 2 use different question sets (text_qa.json is general handbook questions; visual_qa.json is specifically about the tables and forms), so this is not a strict apples-to-apples score - but the numbers still show what each system is good at.

System	Correctness	Avg. Latency
SmolLM2-135M (text, Phase 1)	0.467	0.48 s
Llama 3.2-3B (text, Phase 1)	0.900	12.18 s
gemma3:4b (multimodal, Phase 2)	0.750	15.44 s

- Llama 3.2-3B (text) is the strongest system overall on general handbook questions, but it never sees the table/form pages as images - only as extracted text.
- The multimodal pipeline is answering a harder, table-specific question set. Despite that, it lands between SmolLM2 and Llama, suggesting the table image is genuinely helping on the pages where plain text extraction is known to be garbled.
- SmolLM2 (text) is by far the fastest but the least reliable, confirming the same accuracy/latency trade-off from Phase 1 carries over when compared across both phases.
- A fairer next step, left as future work: re-run the same 12 visual questions through the Llama text-only pipeline, to separate how much of Phase 2's score is the image versus Llama simply being a better reader than SmolLM2.

8. Failure Analysis and Root Causes

Observed Issue	Likely Root Cause	Effect
Gold page sometimes missing from top-4 retrieval	Pages 15-17 (the offence table) are topically near-duplicate at page-level chunking granularity	Lower Context Precision (0.25); both models equally affected, isolating the issue to retrieval
Hallucination on out-of-scope questions	SmolLM2-135M has not learned reliable refusal behaviour at 135M parameters	0/4 correct refusals for SmolLM2 vs 4/4 for Llama
Factual errors on simple lookups with correct context	Limited model capacity / weak instruction-following in SmolLM2-135M	Low Correctness (0.467) despite identical retrieval to Llama
Wrong offence number on a known-defective table row	Row loses its offence number where the table splits across pages 16-17; the VLM guessed rather than reporting it missing	V06 scored 0 despite an image being used
Multimodal answer no better than text on 2/12 questions	Those pages have no pre-made table image, so the VLM falls back to the same garbled text Phase 1 uses	V09, V12 scored 0
Higher Llama latency	Larger model (3B vs 135M parameters)	~25x slower response time (12.18 s vs 0.48 s)

9. Conclusion

The project implemented a reproducible local dual-model RAG pipeline and extended it to a multimodal RAG pipeline, both sharing one retriever so that generation quality could be isolated from retrieval quality. On the 30-question Phase 1 evaluation, retrieval metrics were identical for both models, while Llama 3.2-3B achieved substantially stronger Faithfulness and Correctness, and SmolLM2-135M was ~25x faster. Phase 2 showed that combining retrieved text with a pre-made table image measurably helps on the handbook's table-heavy pages, but only on pages where such an image exists - confirming that both phases remain fundamentally bounded by retrieval quality and, for Phase 2, by image coverage.

10. Reproducibility

- Create the conda environment from environment.yml and pull the required Ollama models (smollm2:135m, llama3.2:3b, gemma3:4b, nomic-embed-text).
- Place the handbook PDF at data/handbook.pdf.
- Run 01_document_processing.ipynb to extract text, render page images, and build the Chroma index.
- Run 02_dual_rag.ipynb for a quick demo of both models on one question.
- Run 03_evaluation.ipynb to regenerate the 30-question Phase 1 evaluation (CSV + summary JSON).
- Run 04_multimodal_rag.ipynb to regenerate the 12-question Phase 2 evaluation and the structured Phase 1 vs Phase 2 comparison.

11. Project Output Files

- smollm_evaluation.csv, llama_evaluation.csv - 30-question Phase 1 evaluation, per question.
- smollm_summary.json, llama_summary.json - aggregate Phase 1 metrics per model.
- multimodal_evaluation.json - 12-question Phase 2 evaluation, per question.
- Architecture diagrams (diagrams/*.mmd), notebooks, and source code (src/).